

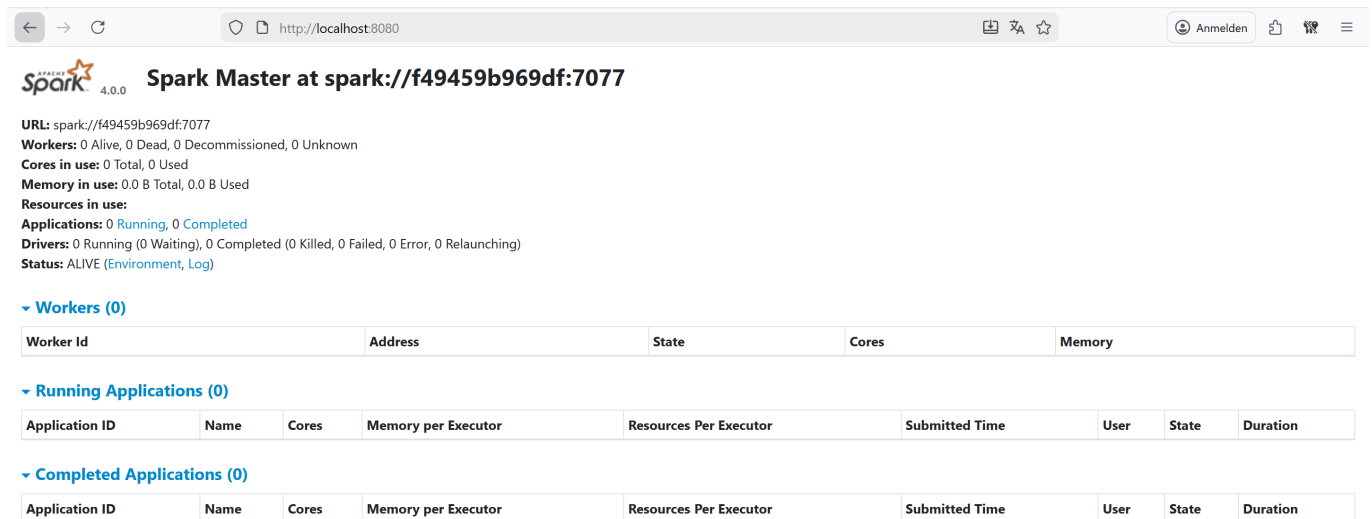
Part 1 Setup

github repo: <https://github.com/smn-krn/SBD3-Part2/tree/main/Exercise3>

in the exercise3 folder

```
docker compose up -d
```

visit <http://localhost:8080/>



The screenshot shows the Spark Master UI. At the top, it says "Spark Master at spark://f49459b969df:7077". Below this, there are statistics: URL, Workers (0 Alive, 0 Dead, 0 Decommissioned, 0 Unknown), Cores in use (0 Total, 0 Used), Memory in use (0.0 B Total, 0.0 B Used), Resources in use, Applications (0 Running, 0 Completed), Drivers (0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)), and Status (ALIVE). There are three expandable sections: "Workers (0)", "Running Applications (0)", and "Completed Applications (0)". Each section has a table with columns: Worker Id, Address, State, Cores, Memory (for Workers); Application ID, Name, Cores, Memory per Executor, Resources Per Executor, Submitted Time, User, State, Duration (for Applications).

At this point the Spark Master UI is available. This confirms that the Spark cluster (master + worker) and Kafka are running correctly inside Docker.

Part 2 - Create Kafka topic

in git bash

```
docker exec -it kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic logs \
  --partitions 2 \
  --replication-factor 1
```

This creates the Kafka topic `logs` which will be used by:

- the **load generator** as producer
 - the **Spark Structured Streaming application** as consumer
-

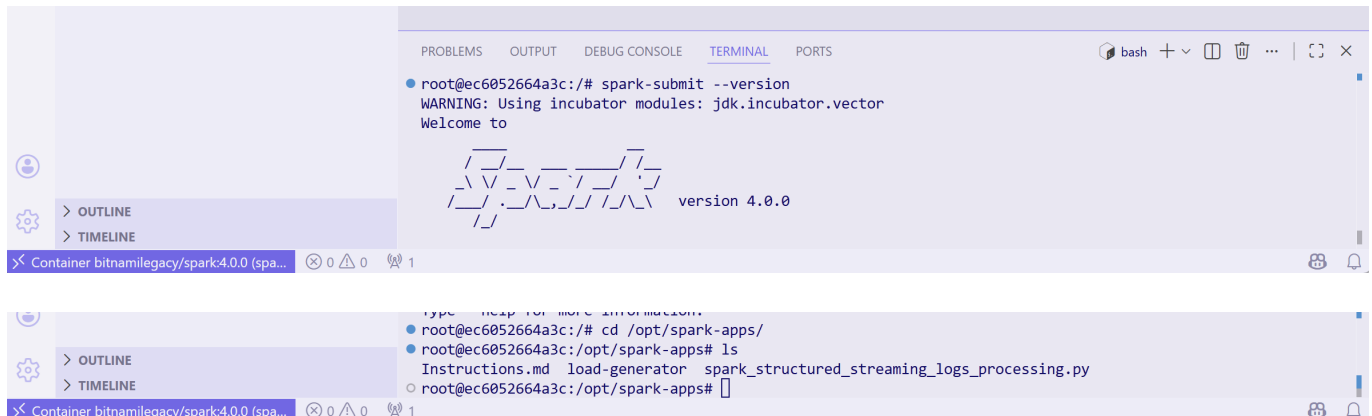
Part 3 - Attach VSCode to Spark

Ctrl + Shift + P

Dev Containers: Attach to Running Container

spark-client

this reopens the vscode => in the new window we get



Now VS Code is attached directly to the `spark-client` container. This ensures:

- correct Spark version (4.0.0)
- correct Kafka networking
- identical environment for execution

Part 4 - Run the Spark Streaming Application

inside the spark-client VSCode

```
spark-submit \  
  --master spark://spark-master:7077 \  
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0 \  
  --num-executors 1 \  
  --executor-cores 1 \  
  --executor-memory 1G \  
  /opt/spark-apps/spark_structured_streaming_logs_processing.py
```

now at <http://localhost:4040> we get

← → ↺

http://localhost:4040/jobs/

🖨 ⚙ ⭐

Anmelden 📄 🧑‍🔬 ☰

SPARK 4.0.0

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

Structured Streaming

LogsProcessor application UI

Spark Jobs (?)

User: spark

Started At: 2026/01/20 19:37:24

Total Uptime: 11 s

Scheduling Mode: FIFO

Active Jobs: 1

▶ Event Timeline

▼ Active Jobs (1)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id (Job Group) ▼	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0 (3faa83bc-0a7d-475b-a5b7-a60ba3797a9f)	id = c54323e1-4e3f-41f8-98fb-c077143648e2 runId = 3faa83bc-0a7d-475b-a5b7-a60ba3797a... start at <unknown>->0 (kill)	2026/01/20 19:37:33	2 s	0/2	0/200

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

and at <http://localhost:8080>

← → ↺

http://localhost:8080

🖨 ⚙ ⭐

Anmelden 📄 🧑‍🔬 ☰

SPARK 4.0.0

Spark Master at spark://f49459b969df:7077

URL: spark://f49459b969df:7077

Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 1 Total, 1 Used

Memory in use: 1024.0 MiB Total, 1024.0 MiB Used

Resources in use:

Applications: 1 Running, 0 Completed

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE ([Environment](#), [Log](#))

▼ Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260120192959-172.20.0.5-37533	172.20.0.5:37533	ALIVE	1 (1 Used)	1024.0 MiB (1024.0 MiB Used)

▼ Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120193726-0000 (kill)	LogsProcessor	1	1024.0 MiB		2026/01/20 19:37:26	root	RUNNING	15 s

▼ Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

At this point the Spark application is running, but no data is processed yet because no events are being produced into Kafka.

Part 5 - Start the load generator

on the normal VSCode

```
cd logs-processing/load-generator
docker compose up -d
```

now at <http://localhost:4040> we get

← → ↺ http://localhost:8080

Spark Master at spark://f49459b969df:7077
4.0.0

URL: spark://f49459b969df:7077

Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 1 Total, 1 Used

Memory in use: 1024.0 MiB Total, 1024.0 MiB Used

Resources in use:

Applications: 1 Running, 0 Completed

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE ([Environment](#), [Log](#))

▼ Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260120192959-172.20.0.5-37533	172.20.0.5:37533	ALIVE	1 (1 Used)	1024.0 MiB (1024.0 MiB Used)

▼ Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120193726-0000 (kill)	LogsProcessor	1	1024.0 MiB		2026/01/20 19:37:26	root	RUNNING	3.6 min

▼ Completed Applications (0)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

Part 6 - Eval (Activity 1)

The aggregation stage involving `groupBy` introduces a shuffle and consistently shows the highest duration in the Spark UI. This is expected because shuffle operations require redistributing data across executors, which is network- and disk-intensive.

4 / 10

The Executors tab shows approximately X MB used out of Y MB available per executor, indicating that the application is CPU-bound rather than memory-bound in the baseline configuration.

Performance & Scalability Concepts

Spark scales by parallelizing tasks across executors and cores. Shuffle operations are the main scalability bottleneck, and efficient partitioning is crucial to avoid data skew and underutilization of resources.

Jobs & DAG

Each micro-batch in Structured Streaming triggers a Spark Job. In the Jobs tab, the DAG visualization shows:

- parsing of Kafka records
- filtering logic
- aggregation stages

Transformations like filtering and projection are executed in the same stage, while aggregations introduce shuffle boundaries.

Stages

In the Stages tab, shuffle read/write can be observed during aggregation. This highlights the cost of redistributing data across executors.

Executors

The Executors tab shows:

- memory usage per executor
- number of active tasks
- CPU utilization

This confirms how Spark parallelizes the workload and helps identify bottlenecks or data skew.

Part 7 - Stop Spark Client & Performance Tuning (Activity 2)

In the spark client window

```
CTRL + C
```

then try to make it perform better via:

```
spark-submit \  
  --master spark://spark-master:7077 \  
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0 \  
  --
```

```
--num-executors 2 \  
--executor-cores 1 \  
--executor-memory 2G \  
/opt/spark-apps/spark_structured_streaming_logs_processing.py
```

I have no idea why but it cannot reach it somehow anymore

LEVIS

Dashboard | Kafka Spark S Azure Machine Le SBD3-Part2/E Login Recruiting Pri Application C LogsProcess Spark Mas X

http://localhost:8080

Anmelden

Spark Master at spark://f49459b969df:7077

URL: spark://f49459b969df:7077
Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown
Cores in use: 1 Total, 0 Used
Memory in use: 1024.0 MiB Total, 0.0 B Used
Resources in use:
Applications: 1 Running, 2 Completed
Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)
Status: ALIVE (Environment, Log)

Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260120192959-172.20.0.5-37533	172.20.0.5:37533	ALIVE	1 (0 Used)	1024.0 MiB (0.0 B Used)

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120194748-0002	(kill) LogsProcessor	0	2.0 GiB		2026/01/20 19:47:48	root	WAITING	1.6 min

Completed Applications (2)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120194601-0001	LogsProcessor	0	2.0 GiB		2026/01/20 19:46:01	root	FINISHED	1.7 min
app-20260120193726-0000	LogsProcessor	1	1024.0 MiB		2026/01/20 19:37:26	root	FINISHED	4.7 min

Jobs Stages Storage Environment Executors SQL / DataFrame Structured Streaming

LogsProcessor application UI

Spark Jobs (?)

User: spark
Started At: 2026/01/20 19:47:47
Total Uptime: 1.6 min
Scheduling Mode: FIFO
Active Jobs: 1

Event Timeline

Active Jobs (1)

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id (Job Group) *	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
0 (3eff8096-ce68-4d28-b3c2-9185138f7234)	id = 667e45e2-ec2b-4a37-83ac-9bdd950a4f31 runid = 3eff8096-ce68-4d28-b3c2-9185138f72... start at <unknown>:0 (kill) 19:47:55	2026/01/20 19:47:55	1.4 min	0/2	0/202

Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go

Tried different configs too; didn't help either. Genuinely I am at a loss.

Increasing the number of executors and executor memory is intended to improve throughput by increasing parallelism. However, in this environment the benefits were limited, likely due to the low number of Kafka partitions and the overhead of shuffle operations dominating execution time.

Part 8 - Change code (Activity 3)

Original code

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, lower, count, desc
from pyspark.sql.types import StructType, StructField, StringType, LongType
...
```

The original implementation:

- filtered on "vulnerability"
- grouped by source_ip
- used processing-time semantics
- did not implement windowing or user-based aggregation

Modified code (Activity 3 implementation)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col,
    from_json,
    lower,
    count,
    window,
    to_timestamp
)
from pyspark.sql.types import (
    StructType,
    StructField,
    StringType,
    LongType
)

CHECKPOINT_PATH = "/tmp/spark-checkpoints/activity-3"

spark = (
    SparkSession.builder
    .appName("Activity3-CrashMonitoring")
    .config("spark.sql.streaming.checkpointLocation", CHECKPOINT_PATH)
    .getOrCreate()
)

spark.sparkContext.setLogLevel("ERROR")

schema = StructType([
    StructField("timestamp", LongType()),
    StructField("status", StringType()),
    StructField("severity", StringType()),
    StructField("source_ip", StringType()),
    StructField("user_id", StringType()),
```

```
    StructField("content", StringType())
  ])

raw_df = (
  spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "kafka:9092")
    .option("subscribe", "logs")
    .option("startingOffsets", "earliest")
    .option("failOnDataLoss", "false")
    .load()
)

parsed_df = (
  raw_df
    .select(from_json(col("value").cast("string"), schema).alias("data"))
    .select("data.*")
)

events_df = parsed_df.withColumn(
  "event_time",
  to_timestamp(col("timestamp"))
)

filtered_df = (
  events_df
    .filter(lower(col("content")).contains("crash"))
    .filter(col("severity").isin("High", "Critical"))
)

aggregated_df = (
  filtered_df
    .withWatermark("event_time", "20 seconds")
    .groupBy(
      window(col("event_time"), "10 seconds"),
      col("user_id")
    )
    .agg(count("*").alias("crash_count"))
    .filter(col("crash_count") > 2)
)

query = (
  aggregated_df
    .writeStream
    .outputMode("append")
    .format("console")
    .option("truncate", "false")
    .start()
)

query.awaitTermination()
```


Event-time & Windowing

The application uses **event-time processing** based on the `timestamp` field contained in the log record. A **10-second tumbling window** is applied to aggregate crash events per `user_id`.

Watermark & Late Data

A watermark of **20 seconds** is defined:

```
.withWatermark("event_time", "20 seconds")
```

This allows the system to handle **late-arriving events** while keeping state bounded. Events arriving later than the watermark are safely discarded.

Scalability & Fault Tolerance

- Kafka partitions enable parallel consumption
- Spark executors scale horizontally
- Checkpointing ensures recovery after failures
- Stateful operations are safely recovered using the checkpoint directory

Re-execution

```
spark-submit \  
  --master spark://spark-master:7077 \  
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0 \  
  --num-executors 1 \  
  --executor-cores 1 \  
  --executor-memory 1G \  
  /opt/spark-apps/spark_structured_streaming_logs_processing.py
```

and then

```
docker compose up -d
```

←→↺

http://localhost:8080

Anmelden

spark

4.0.0

Spark Master at spark://f49459b969df:7077

URL: spark://f49459b969df:7077

Workers: 1 Alive, 0 Dead, 0 Decommissioned, 0 Unknown

Cores in use: 1 Total, 1 Used

Memory in use: 1024.0 MiB Total, 1024.0 MiB Used

Resources in use:

Applications: 1 Running, 3 Completed

Drivers: 0 Running (0 Waiting), 0 Completed (0 Killed, 0 Failed, 0 Error, 0 Relaunching)

Status: ALIVE ([Environment](#), [Log](#))

▼ Workers (1)

Worker Id	Address	State	Cores	Memory
worker-20260120192959-172.20.0.5-37533	172.20.0.5:37533	ALIVE	1 (1 Used)	1024.0 MiB (1024.0 MiB Used)

▼ Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120195341-0003	(kill) Activity3-CrashMonitoring	1	1024.0 MiB		2026/01/20 19:53:41	root	RUNNING	51 s

▼ Completed Applications (3)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20260120194748-0002	LogsProcessor	0	2.0 GiB		2026/01/20 19:47:48	root	FINISHED	2.9 min
app-20260120194601-0001	LogsProcessor	0	2.0 GiB		2026/01/20 19:46:01	root	FINISHED	1.7 min
app-20260120193726-0000	LogsProcessor	1	1024.0 MiB		2026/01/20 19:37:26	root	FINISHED	4.7 min

←→↺

http://localhost:4040/stages/

Anmelden

spark

4.0.0

Jobs

Stages

Storage

Environment

Executors

SQL / DataFrame

Structured Streaming

Activity3-CrashMonitoring application UI

Stages for All Jobs

Active Stages: 1

Completed Stages: 3

▼ Active Stages (1)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
3	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>>0 +details (kill)	2026/01/20 19:55:11	8 s	104/200 (2 running)			44.7 KiB	

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

▼ Completed Stages (3)

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
2	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>>0 +details	2026/01/20 19:55:07	4 s	2/2				86.1 KiB
1	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>>0 +details	2026/01/20 19:54:45	21 s	200/200			538.5 KiB	
0	id = 6caa0e1a-5579-477f-8c1c-25d778e88593 runId = 3487c4b5-de2c-4ca9-af8c-9dab07d905c9 b... start at <unknown>>0 +details	2026/01/20 19:53:49	56 s	2/2				538.5 KiB

Page: 1

1 Pages. Jump to 1 . Show 100 items in a page. Go

10 / 10